Read the slide first [slide link]

Adjacency matrix

Types 4:

- Unweighted (show connections with 1)
  - Undirected (u to v, v to u)
  - Directed (u to v)
- Weighted (show connections with edge weight)
  - Undirected (u to v, v to u)
  - Directed (u to v)

Adjacency matrix for Undirected Unweighted Graph

```
public class GraphUndirectedUnweightedMatrix {

    private int numVertices;    // Total number of vertices
    private int[][] adjMatrix;  // 2D matrix to store edges

    // Constructor: initialize graph
    public GraphUndirectedUnweightedMatrix(int numVertices) {
        this.numVertices = numVertices;

        // Create a V x V matrix
        // Initially all values are 0 (no edges)
        adjMatrix = new int[numVertices][numVertices];
    }

    // Method to add edges (UNDIRECTED)
    public void addEdges(int[][] edges) {

        // edges format: { {source, destination}, ... }
        for (int i = 0; i < edges.length; i++) {

            int source = edges[i][0];        // one vertex
            int destination = edges[i][1];   // other vertex

            // For UNDIRECTED graph:
            // Add edge in BOTH directions
            adjMatrix[source][destination] = 1;
            adjMatrix[destination][source] = 1;
        }
    }
```

```java
// Method to print adjacency matrix
public void printMatrix() {

    System.out.println("Adjacency Matrix:");

    // Traverse rows
    for (int i = 0; i < numVertices; i++) {

        // Traverse columns
        for (int j = 0; j < numVertices; j++) {

            // Print 0 or 1
            System.out.print(adjMatrix[i][j] + " ");
        }

        System.out.println(); // New line after each row
    }
}

// Method to find and print neighbors of a given vertex
public void findNeighbors(int vertex) {

    System.out.print("\nNeighbors of vertex " + vertex + ": ");

    // Traverse row = vertex
    for (int i = 0; i < numVertices; i++) {

        // If there is an edge vertex – i
        if (adjMatrix[vertex][i] == 1) {

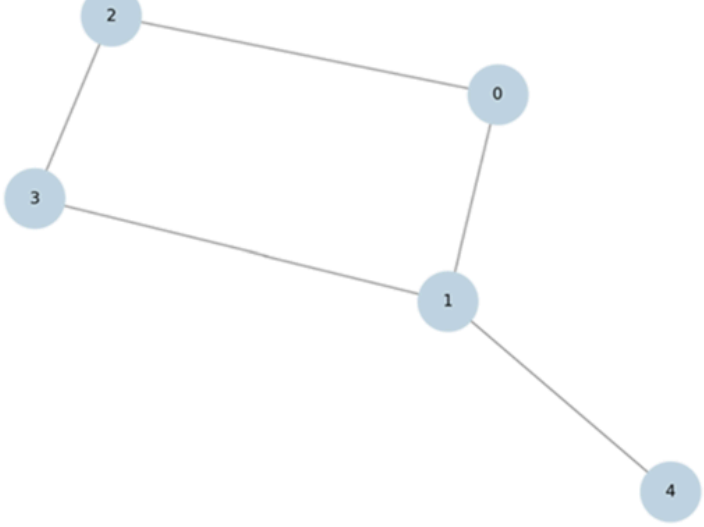
            // Print neighbor
            System.out.print(i + "  ");
        }
    }

    System.out.println();
}
```

Python

↳ 1 cell hidden

Driver



	0	1	2	3	4
0	0	1	1	0	0
1	1	0	0	1	1
2	1	0	0	1	0
3	0	1	1	0	0
4	0	1	0	0	0

```
Adjacency List:
0: [1, 2]
1: [0, 3, 4]
2: [0, 3]
3: [1, 2]
4: [1]
```

```java
// Driver code
public static void main(String[] args) {
    int numVertices = 5;
    int[][] edges1 = {
        {0, 1}, {0, 2},
        {1, 3}, {1, 4},
        {2, 3}
    };

    GraphUndirectedMatrix graph1 = new GraphUndirectedMatrix(numVertices);
    System.out.println("Before putting connections:");
    graph1.printMatrix();

    graph1.addEdges(edges1);
    System.out.println("\nAfter putting connections:");
    graph1.printMatrix();

    for (int i = 0; i < numVertices; i++) {
        graph1.findNeighbors(i);
    }
```

```python
# Driver code
num_vertices = 5
edges = [
    (0, 1),  # (U, V) = U=START_POINT. V=END_POINT
    (0, 2),
    (1, 3),
    (1, 4),
    (2, 3),
]

# Create graph instance
graph = Graph(num_vertices)

print("Before putting connections:")
graph.print_matrix()

# Add edges
graph.add_edges(edges)

print("\nAfter putting connections:")
graph.print_matrix()

# Find neighbors for each vertex
graph.find_neighbors(0)
graph.find_neighbors(1)
graph.find_neighbors(2)
graph.find_neighbors(3)
graph.find_neighbors(4)
```

```
Before putting connections:
Adjacency Matrix:
[0 0 0 0 0]
[0 0 0 0 0]
[0 0 0 0 0]
[0 0 0 0 0]
[0 0 0 0 0]

After putting connections:
Adjacency Matrix:
[0 1 1 0 0]
[1 0 0 1 1]
[1 0 0 1 0]
[0 1 1 0 0]
[0 1 0 0 0]

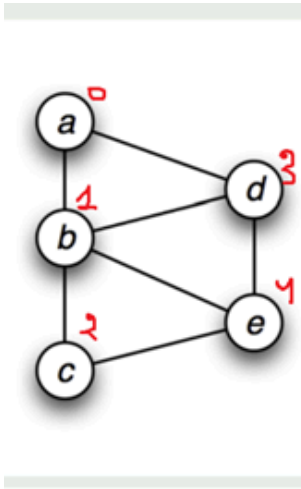
Neighbors of vertex 0: 12

Neighbors of vertex 1: 034

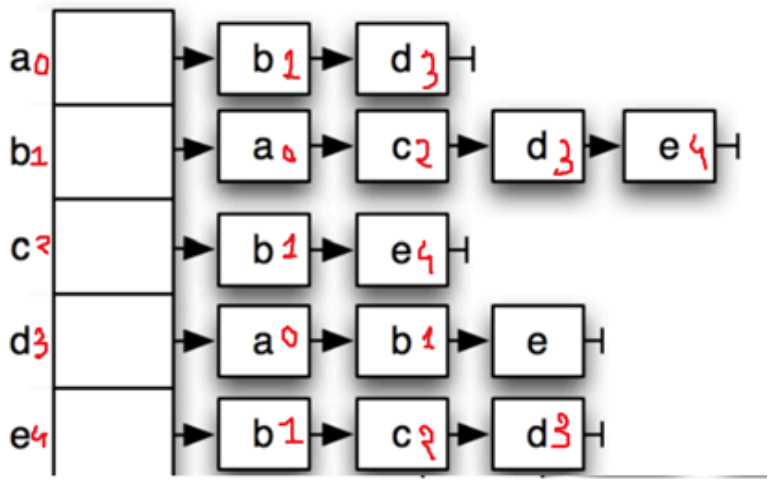
Neighbors of vertex 2: 03
```

```
Neighbors of vertex 3: 12

Neighbors of vertex 4: 1
```



	a	b	c	d	e
0 a	0	1	0	1	0
1 b	1	0	1	1	1
2 c	0	1	0	0	1
3 d	1	1	0	0	1
4 e	0	1	1	1	0




```
    int[][] edges2 = {
        {0, 1}, {0, 3},
        {1, 3}, {1, 2}, {1, 4},
        {2, 4},
        {3, 4}
    };

    GraphUndirectedMatrix graph2 = new GraphUndirectedMatrix(numVertices);
    System.out.println("Before putting connections:");
    graph2.printMatrix();

    graph2.addEdges(edges2);
    System.out.println("\nAfter putting connections:");
    graph2.printMatrix();

    for (int i = 0; i < numVertices; i++) {
        graph2.findNeighbors(i);
    }
}
```

```
# Driver code
num_vertices = 5
edges = [
    (0, 1), (0, 3),
    (1, 3), (1, 2), (1, 4),
    (2, 4),
    (3, 4)
]

# Create an instance of the Graph class
graph = Graph(num_vertices)

print("Before putting connections:")
graph.print_matrix()

# Add edges to the graph
graph.add_edges(edges)

print("\nAfter putting connections:")
graph.print_matrix()

# Find neighbors for each vertex
graph.find_neighbors(0)
graph.find_neighbors(1)
graph.find_neighbors(2)
graph.find_neighbors(3)
graph.find_neighbors(4)
```

```
Before putting connections:
Adjacency Matrix:
```

```
[0 0 0 0 0]
[0 0 0 0 0]
[0 0 0 0 0]
[0 0 0 0 0]
[0 0 0 0 0]

After putting connections:
Adjacency Matrix:
[0 1 0 1 0]
[1 0 1 1 1]
[0 1 0 0 1]
[1 1 0 0 1]
[0 1 1 1 0]

Neighbors of vertex 0: 13

Neighbors of vertex 1: 0234

Neighbors of vertex 2: 14

Neighbors of vertex 3: 014

Neighbors of vertex 4: 123
```

Adjacency matrix for Directed Unweighted Graph

```java
public class GraphDirectedUnweightedMatrix {

    private int numVertices;    // Total number of vertices
    private int[][] adjMatrix;  // 2D matrix to store edges

    // Constructor: initialize graph
    public GraphDirectedUnweightedMatrix(int numVertices) {
        this.numVertices = numVertices;

        // Create a V x V matrix
        // Initially all values are 0 (no edges)
        adjMatrix = new int[numVertices][numVertices];
    }

    // Add multiple edges (DIRECTED)
    public void addEdges(int[][] edges) {

        // edges format: { {source, destination}, ... }
        for (int i = 0; i < edges.length; i++) {

            int source = edges[i][0];        // starting vertex
            int destination = edges[i][1];   // ending vertex

            // For DIRECTED graph:
            // Mark only source → destination
            adjMatrix[source][destination] = 1;
        }
    }
```

```java
    // Print the adjacency matrix
    public void printMatrix() {

        System.out.println("Adjacency Matrix:");

        // Traverse rows
        for (int i = 0; i < numVertices; i++) {

            // Traverse columns
            for (int j = 0; j < numVertices; j++) {

                // Print 0 or 1
                System.out.print(adjMatrix[i][j] + " ");
            }

            System.out.println(); // New line after each row
        }
    }

    // Print neighbors of a given vertex
    public void findNeighbors(int vertex) {

        System.out.print("\nNeighbors of vertex " + vertex + ": ");

        // Traverse row = vertex
        for (int i = 0; i < numVertices; i++) {

            // If there is an edge vertex → i
            if (adjMatrix[vertex][i] == 1) {

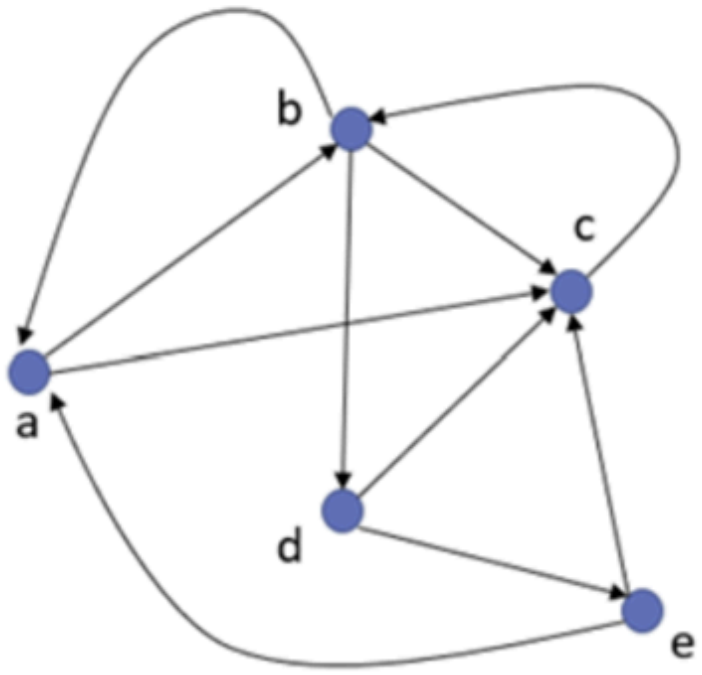
                // Print neighbor
                System.out.print(i + "  ");
            }
        }

        System.out.println();
    }
```

Python

↳ 1 cell hidden

Driver



	a 0	b 1	c 2	d 3	e 4
a 0	0	1	1	0	0
b 1	1	0	1	1	0
c 2	0	1	0	0	0
d 3	0	0	1	0	1
e 4	1	0	1	0	0

```java
// Driver code
public static void main(String[] args) {
    int numVertices = 5;
    int[][] edges = {
        {0, 1}, {0, 2},
        {1, 0}, {1, 2}, {1, 3},
        {2, 1},
        {3, 2}, {3, 4},
        {4, 0}, {4, 2}
    };

    GraphDirectedMatrix graph = new GraphDirectedMatrix(numVertices);

    System.out.println("Before putting connections:");
    graph.printMatrix();

    System.out.println("\nAfter putting connections:");
    graph.addEdges(edges);
    graph.printMatrix();

    for (int i = 0; i < numVertices; i++) {
        graph.findNeighbors(i);
    }
}
```

```python
# Driver code
num_vertices = 5
edges = [
    (0, 1), (0, 2),
    (1, 0), (1, 2), (1, 3),
    (2, 1),
    (3, 2), (3, 4),
    (4, 0), (4, 2)
]

# Create an instance of the Graph class
graph = Graph(num_vertices)

print("Before putting connections:")
graph.print_matrix()

# Add edges to the graph
graph.add_edges(edges)

print("\nAfter putting connections:")
graph.print_matrix()

# Find neighbors for each vertex
graph.find_neighbors(0)
graph.find_neighbors(1)
graph.find_neighbors(2)
graph.find_neighbors(3)
graph.find_neighbors(4)
```

```
Before putting connections:
Adjacency Matrix:
[0 0 0 0 0]
[0 0 0 0 0]
[0 0 0 0 0]
[0 0 0 0 0]
[0 0 0 0 0]

After putting connections:
Adjacency Matrix:
[0 1 1 0 0]
[1 0 1 1 0]
[0 1 0 0 0]
[0 0 1 0 1]
```

```
[1 0 1 0 0]

Neighbors of vertex 0: 1 2

Neighbors of vertex 1: 0 2 3

Neighbors of vertex 2: 1

Neighbors of vertex 3: 2 4

Neighbors of vertex 4: 0 2
```

Adjacency matrix for Directed Weighted graph

```
public class GraphDirectedWeightedMatrix {

    private int numVertices;    // Total number of vertices
    private int[][] adjMatrix;  // 2D matrix to store edge weights

    // Constructor: initialize graph
    public GraphDirectedWeightedMatrix(int numVertices) {
        this.numVertices = numVertices;

        // Create a V x V matrix
        // Initially all values are 0 (means no edge)
        adjMatrix = new int[numVertices][numVertices];
    }

    // Add multiple edges with weights (DIRECTED)
    public void addEdges(int[][] edges) {

        // edges format: { {source, destination, weight}, ... }
        for (int i = 0; i < edges.length; i++) {

            int source = edges[i][0];        // starting vertex
            int destination = edges[i][1];   // ending vertex
            int weight = edges[i][2];        // weight of the edge

            // For DIRECTED graph:
            // Store weight only for source → destination
            adjMatrix[source][destination] = weight;
        }
    }
```

```java
    // Print the adjacency matrix
    public void printMatrix() {

        System.out.println("Adjacency Matrix:");

        // Traverse rows
        for (int i = 0; i < numVertices; i++) {

            // Traverse columns
            for (int j = 0; j < numVertices; j++) {

                // Print weight (0 means no edge)
                System.out.print(adjMatrix[i][j] + " ");
            }

            System.out.println(); // New line after each row
        }
    }

    // Print neighbors of a given vertex
    public void findNeighbors(int vertex) {

        System.out.print("\nNeighbors of vertex " + vertex + ": ");

        // Traverse row = vertex
        for (int i = 0; i < numVertices; i++) {

            // If weight is non-zero → edge exists
            if (adjMatrix[vertex][i] != 0) {

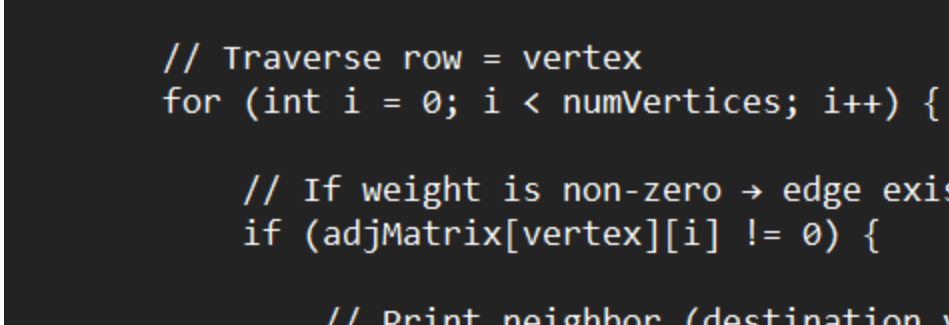
                // Print neighbor (destination vertex)
                System.out.print(i + " ");
            }
        }

        System.out.println();
    }
```

Python

↳ 1 cell hidden

Driver



	0	1	2	3	4
0	0	2	3	0	0
1	0	0	0	4	5
2	0	0	0	6	0
3	0	0	0	0	0
4	0	0	0	0	0

```java
// Driver code
public static void main(String[] args) {
    int numVertices = 5;
    int[][] edges = {
        {0, 1, 2}, // Edge from 0 to 1 with weight 2
        {0, 2, 3}, // Edge from 0 to 2 with weight 3
        {1, 3, 4}, // Edge from 1 to 3 with weight 4
        {1, 4, 5}, // Edge from 1 to 4 with weight 5
        {2, 3, 6}  // Edge from 2 to 3 with weight 6
    };

    GraphDirectedWeightedMatrix graph = new GraphDirectedWeightedMatrix(numVertices);

    System.out.println("Before putting connections:");
    graph.printMatrix();

    System.out.println("\nAfter putting connections:");
    graph.addEdges(edges);
    graph.printMatrix();

    for (int i = 0; i < numVertices; i++) {
        graph.findNeighbors(i);
    }
}
```

```python
# Driver code
num_vertices = 5
edges = [
    (0, 1, 2),  # Edge from 0 to 1 with weight 2
    (0, 2, 3),  # Edge from 0 to 2 with weight 3
    (1, 3, 4),  # Edge from 1 to 3 with weight 4
    (1, 4, 5),  # Edge from 1 to 4 with weight 5
    (2, 3, 6),  # Edge from 2 to 3 with weight 6
]

# Create an instance of the Graph class
graph = Graph(num_vertices)

print("Before putting connections:")
graph.print_matrix()

# Add weighted edges to the graph
graph.add_edges(edges)

print("\nAfter putting connections:")
graph.print_matrix()

# Find and print neighbors for each vertex
graph.find_neighbors(0)
graph.find_neighbors(1)
graph.find_neighbors(2)
graph.find_neighbors(3)
graph.find_neighbors(4)
```

```
Before putting connections:
Adjacency Matrix:
[0 0 0 0 0]
[0 0 0 0 0]
[0 0 0 0 0]
[0 0 0 0 0]
[0 0 0 0 0]

After putting connections:
Adjacency Matrix:
[0 2 3 0 0]
[0 0 0 4 5]
[0 0 0 6 0]
[0 0 0 0 0]
[0 0 0 0 0]

Neighbors of vertex 0: 1  2

Neighbors of vertex 1: 3  4

Neighbors of vertex 2: 3
```

```
Neighbors of vertex 3:

Neighbors of vertex 4:
```

Adjacency matrix for undirected Weighted graph

```java
public class GraphUndirectedWeightedMatrix {

    private int numVertices;    // Total number of vertices
    private int[][] adjMatrix;  // 2D matrix to store edge weights

    // Constructor: initialize graph
    public GraphUndirectedWeightedMatrix(int numVertices) {
        this.numVertices = numVertices;

        // Create a V x V matrix
        // Initially all values are 0 (means no edge)
        adjMatrix = new int[numVertices][numVertices];
    }

    // Add multiple edges with weights (UNDIRECTED)
    public void addEdges(int[][] edges) {

        // edges format: { {source, destination, weight}, ... }
        for (int i = 0; i < edges.length; i++) {

            int source = edges[i][0];        // one vertex
            int destination = edges[i][1];   // other vertex
            int weight = edges[i][2];        // weight of the edge

            // For UNDIRECTED graph:
            // Store weight in BOTH directions
            adjMatrix[source][destination] = weight;   // source → destination
            adjMatrix[destination][source] = weight;   // destination → source
        }
    }
```

```java
    // Print the adjacency matrix
    public void printMatrix() {

        System.out.println("Adjacency Matrix:");

        // Traverse rows
        for (int i = 0; i < numVertices; i++) {

            // Traverse columns
            for (int j = 0; j < numVertices; j++) {

                // Print weight (0 means no edge)
                System.out.print(adjMatrix[i][j] + " ");
            }

            System.out.println(); // New line after each row
        }
    }

    // Print neighbors of a given vertex
    public void findNeighbors(int vertex) {

        System.out.print("\nNeighbors of vertex " + vertex + ": ");

        // Traverse row = vertex
        for (int i = 0; i < numVertices; i++) {

            // If weight is non-zero → edge exists
            if (adjMatrix[vertex][i] != 0) {

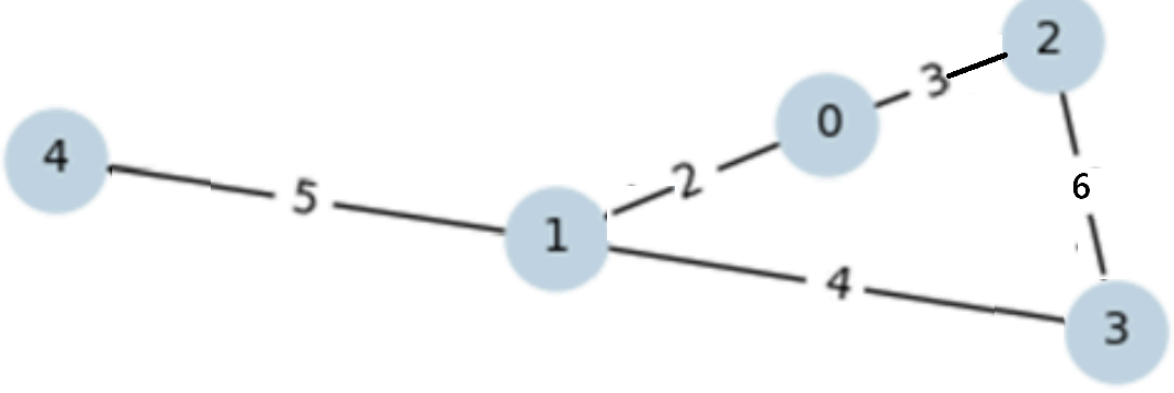
                // Print neighbor (connected vertex)
                System.out.print(i + " ");
            }
        }

        System.out.println();
    }
```

Python

↳ 1 cell hidden

Driver




```
   0  1  2  3  4
0 [0, 2, 3, 0, 0]
1 [2, 0, 0, 4, 5]
2 [3, 0, 0, 6, 0]
3 [0, 4, 6, 0, 0]
4 [0, 5, 0, 0, 0]
```

```java
// Driver code
public static void main(String[] args) {
    int numVertices = 5;
    int[][] edges = {
        {0, 1, 2}, // Edge between 0 and 1 with weight 2
        {0, 2, 3}, // Edge between 0 and 2 with weight 3
        {1, 3, 4}, // Edge between 1 and 3 with weight 4
        {1, 4, 5}, // Edge between 1 and 4 with weight 5
        {2, 3, 6}  // Edge between 2 and 3 with weight 6
    };

    GraphUndirectedWeightedMatrix graph = new GraphUndirectedWeightedMatrix(numVertices);

    System.out.println("Before putting connections:");
    graph.printMatrix();

    System.out.println("\nAfter putting connections:");
    graph.addEdges(edges);
    graph.printMatrix();

    for (int i = 0; i < numVertices; i++) {
        graph.findNeighbors(i);
    }
}
```

```python
# Driver code
num_vertices = 5
edges = [
    (0, 1, 2),  # Edge from 0 to 1 with weight 2
    (0, 2, 3),  # Edge from 0 to 2 with weight 3
    (1, 3, 4),  # Edge from 1 to 3 with weight 4
    (1, 4, 5),  # Edge from 1 to 4 with weight 5
    (2, 3, 6),  # Edge from 2 to 3 with weight 6
]

graph = Undirected_Weighted_AdjacencyMatrix(num_vertices)

print("Before putting connections:")
graph.print_matrix()

graph.add_edges(edges)

print("\nAfter putting connections:")
graph.print_matrix()

for vertex in range(num_vertices):
    graph.find_neighbors(vertex)
```

```
Before putting connections:
Adjacency Matrix:
[0 0 0 0 0]
[0 0 0 0 0]
[0 0 0 0 0]
[0 0 0 0 0]
[0 0 0 0 0]

After putting connections:
Adjacency Matrix:
[0 2 3 0 0]
[2 0 0 4 5]
[3 0 0 6 0]
[0 4 6 0 0]
[0 5 0 0 0]

Neighbors of vertex 0: 1  2

Neighbors of vertex 1: 0  3  4

Neighbors of vertex 2: 0  3

Neighbors of vertex 3: 1  2

Neighbors of vertex 4: 1
```

Adjacency list

Types 4:

- Unweighted
  - Undirected (u to v, v to u)
  - Directed (u to v)
- Weighted
  - Undirected (u to v, v to u)
  - Directed (u to v)

Adjacency list for Undirected unWeighted graph

```
class Edge {
    int destination;
    Edge next;

    public Edge(int destination) {
        this.destination = destination;
        this.next = null;
    }
}
```

```
public class GraphUndirectedUnweightedAdjacencyList {

    private int numVertices;   // Total number of vertices
    private Edge[] adjList;    // Array of linked lists (Adjacency List)

    // Constructor: initialize graph
    public GraphUndirectedUnweightedAdjacencyList(int numVertices) {
        this.numVertices = numVertices;

        // Create an array of size = numVertices
        // Each index will store the head of a linked list
        adjList = new Edge[numVertices]; // Initially all values are null
    }

    // Add multiple edges to the graph
    public void addEdges(int[][] edges) {

        // edges format: { {source, destination}, ... }
        for (int i = 0; i < edges.length; i++) {

            int source = edges[i][0];        // Starting vertex
            int destination = edges[i][1];   // Ending vertex

            // For UNDIRECTED graph:
            // Add edge in BOTH directions
            addEdge(source, destination);      // source → destination
            addEdge(destination, source);      // destination → source
        }
    }

    // Helper method to add a single edge
    private void addEdge(int source, int destination) {

        // Step 1: Create a new edge node
        Edge newEdge = new Edge(destination);

        // Step 2: Insert at the beginning (prepend)
        // This ensures O(1) insertion time
        newEdge.next = adjList[source];

        // Step 3: Update head of the list
        adjList[source] = newEdge;
    }
```

```java
    // Print the full adjacency list
    public void printList() {

        System.out.println("Adjacency List:");

        // Traverse each vertex
        for (int i = 0; i < numVertices; i++) {

            System.out.print(i + ": ");

            // Traverse linked list of edges for vertex i
            Edge current = adjList[i];

            while (current != null) {

                // Print neighbor (destination only, since unweighted)
                System.out.print(current.destination + " ");

                current = current.next; // Move to next edge
            }

            System.out.println();
        }
    }

    // Print neighbors of a given vertex
    public void findNeighbors(int vertex) {

        System.out.print("\nNeighbors of vertex " + vertex + ": ");

        Edge current = adjList[vertex];

        // Traverse all neighbors
        while (current != null) {

            // Print current neighbor
            System.out.print(current.destination);

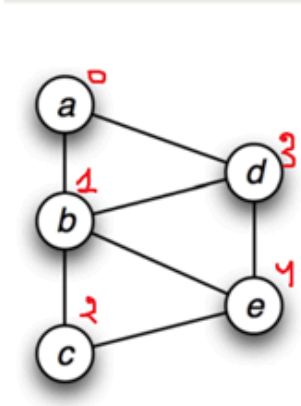
            // Add comma if not the last element
            if (current.next != null) {
                System.out.print(",");
            }

            current = current.next; // Move to next edge
        }
```

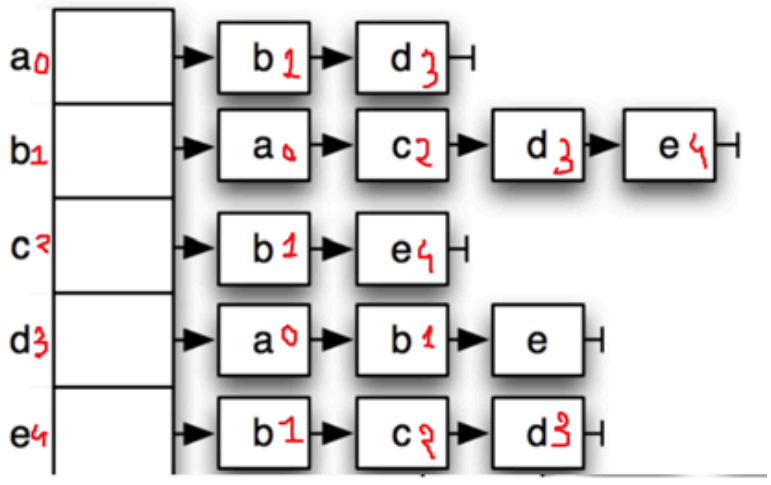
Python

↳ 1 cell hidden

Driver



	0 a	1 b	2 c	3 d	4 e
0 a	0	1	0	1	0
1 b	1	0	1	1	1
2 c	0	1	0	0	1
3 d	1	1	0	0	1
4 e	0	1	1	1	0



```java
public static void main(String[] args) {
    int numVertices = 5;
    int[][] edges = {
        {0, 1},
        {0, 3},
        {1, 3},
        {1, 2},
        {1, 4},
        {2, 4},
        {3, 4}
    };

    GraphUndirectedAdjacencyList graph = new GraphUndirectedAdjacencyList(numVertices);

    System.out.println("Before adding connections:");
    graph.printList();

    graph.addEdges(edges);

    System.out.println("\nAfter adding connections:");
    graph.printList();

    for (int vertex = 0; vertex < numVertices; vertex++) {
        graph.findNeighbors(vertex);
    }
}
```

```python
# Driver code
num_vertices = 5
edges = [
    (0, 1),  # u, v
    (0, 3),
    (1, 3),
    (1, 2),
    (1, 4),
    (2, 4),
    (3, 4)
]

# Create an instance of the AdjacencyListGraph class
graph = AdjacencyListGraph(num_vertices)

print("Before adding connections:")
graph.print_list()

# Add edges to the adjacency list
graph.add_edges(edges)

print("\nAfter adding connections:")
graph.print_list()

# Find neighbors for each vertex
for vertex in range(num_vertices):
    graph.find_neighbors(vertex)

```

```
Before adding connections:
Adjacency List:
0:
1:
2:
3:
4:

After adding connections:
Adjacency List:
0: 3 1
1: 4 2 3 0
2: 4 1
3: 4 1 0
4: 3 2 1

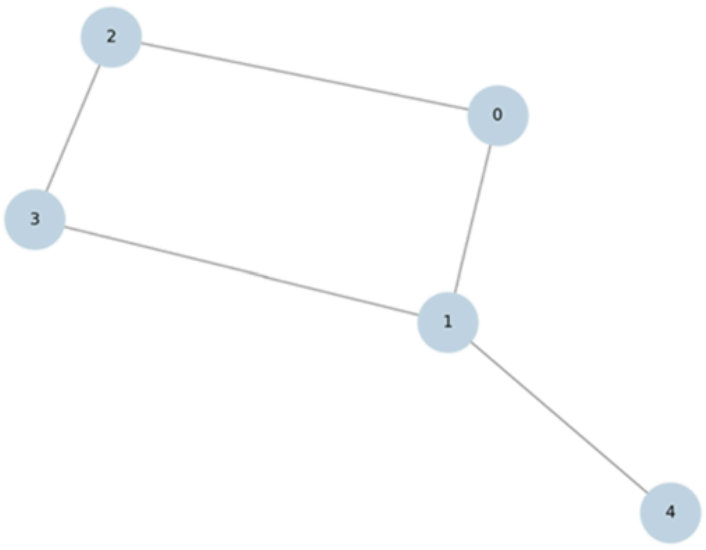
Neighbors of vertex 0: 3,1
```

```
Neighbors of vertex 1: 4,2,3,0

Neighbors of vertex 2: 4,1

Neighbors of vertex 3: 4,1,0

Neighbors of vertex 4: 3,2,1
```



	0	1	2	3	4
0	0	1	1	0	0
1	1	0	0	1	1
2	1	0	0	1	0
3	0	1	1	0	0
4	0	1	0	0	0

Adjacency List:
0: [1, 2]
1: [0, 3, 4]
2: [0, 3]
3: [1, 2]
4: [1]

```python
# Driver code
num_vertices = 5
edges = [
    (0, 1),  # u, v
    (0, 3),
    (1, 3),
    (1, 2),
    (1, 4),
    (2, 4),
    (3, 4)
]

# Create an instance of the AdjacencyListGraph class
graph = AdjacencyListGraph(num_vertices)

# Print the graph before adding connections
print("Before adding connections:")
graph.print_list()

# Add edges to the adjacency list
graph.add_edges(edges)

# Print the graph after adding connections
print("\nAfter adding connections:")
graph.print_list()

# Find and print neighbors for each vertex
for vertex in range(num_vertices):
    graph.find_neighbors(vertex)
```

```
Before adding connections:
Adjacency List:
0:
1:
2:
3:
4:

After adding connections:
Adjacency List:
0: 3 1
1: 4 2 3 0
2: 4 1
3: 4 1 0
4: 3 2 1

Neighbors of vertex 0: 3,1

Neighbors of vertex 1: 4,2,3,0

Neighbors of vertex 2: 4,1

Neighbors of vertex 3: 4,1,0

Neighbors of vertex 4: 3,2,1
```

Adjacency list for Directed unWeighted graph

```
public class GraphDirectedUnweightedAdjacencyList {

    private int numVertices;   // Total number of vertices in the graph
    private Edge[] adjList;    // Array of linked lists (Adjacency List)

    // Constructor: initialize graph
    public GraphDirectedUnweightedAdjacencyList(int numVertices) {
        this.numVertices = numVertices;

        // Create an array of size = numVertices
        // Each index will store the head of a linked list
        adjList = new Edge[numVertices]; // Initially all values are null
    }

    // Add multiple edges to the graph
    public void addEdges(int[][] edges) {

        // edges format: { {source, destination}, ... }
        for (int i = 0; i < edges.length; i++) {

            int source = edges[i][0];        // Starting vertex
            int destination = edges[i][1];   // Ending vertex

            // For DIRECTED graph:
            // Add edge in only ONE direction
            addEdge(source, destination);
        }
    }

    // Helper method to add a single directed edge
    private void addEdge(int source, int destination) {

        // Step 1: Create a new edge node
        Edge newEdge = new Edge(destination);

        // Step 2: Insert at the beginning (prepend)
        // This ensures O(1) insertion time
        newEdge.next = adjList[source];

        // Step 3: Update the head of the list
        adjList[source] = newEdge;
    }
```

```java
    // Print the full adjacency list
    public void printList() {

        System.out.println("Adjacency List:");

        // Traverse each vertex
        for (int i = 0; i < numVertices; i++) {

            System.out.print(i + ": ");

            // Traverse linked list of edges for vertex i
            Edge current = adjList[i];

            while (current != null) {

                // Print only destination (unweighted graph)
                System.out.print(current.destination + " ");

                current = current.next; // Move to next edge
            }
        }
    }

    // Method to find and print neighbors of a given vertex
    public void findNeighbors(int vertex) {
        System.out.print("\nNeighbors of vertex " + vertex + ": ");

        Edge current = adjList[vertex];

        // Traverse all neighbors
        while (current != null) {

            // Print current neighbor
            System.out.print(current.destination);

            // Add comma if not the last element
            if (current.next != null) {
                System.out.print(",");
            }

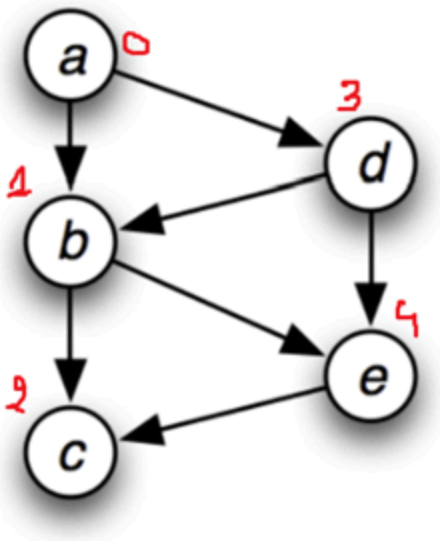
            current = current.next; // Move to next edge
        }

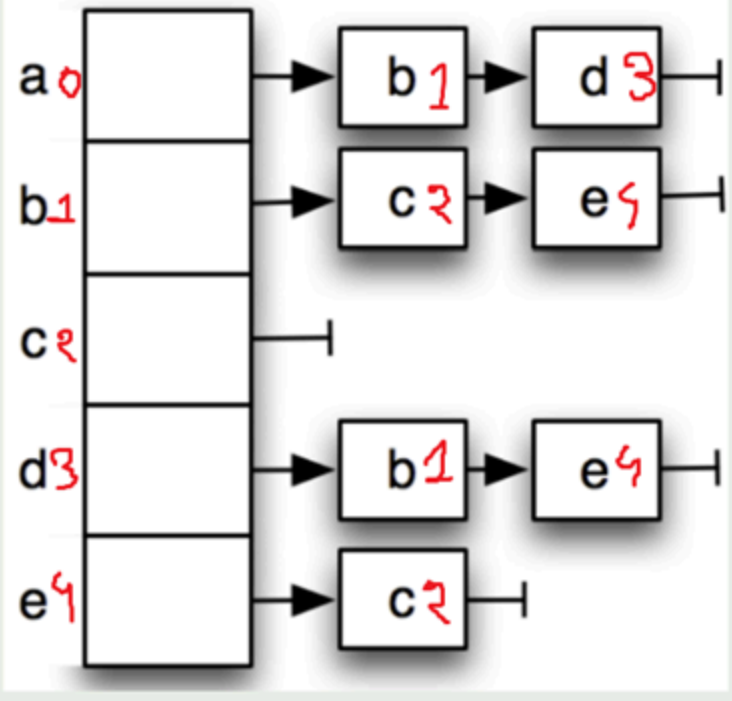
        System.out.println();
    }
```

Python

↳ 1 cell hidden

Driver





```java
// Driver code
public static void main(String[] args) {
    int numVertices = 5;
    int[][] edges = {
        {0, 1}, {0, 3},
        {1, 2}, {1, 4},
        {3, 1}, {3, 4},
        {4, 2}
    };

    GraphDirectedAdjacencyList graph = new GraphDirectedAdjacencyList(numVertices);

    System.out.println("Before adding connections:");
    graph.printList();

    graph.addEdges(edges);

    System.out.println("\nAfter adding connections:");
    graph.printList();

    for (int vertex = 0; vertex < numVertices; vertex++) {
        graph.findNeighbors(vertex);
    }
}
```

```python
# Driver code
num_vertices = 5
edges = [
    (0, 1), (0, 3),
    (1, 2), (1, 4),
    (3, 1), (3, 4),
    (4, 2)
]

# Create an instance of the graph
graph = AdjacencyListGraph(num_vertices)

print("Before adding connections:")
graph.print_list()

# Add undirected edges
graph.add_edges(edges)

print("\nAfter adding connections:")
graph.print_list()

# Print neighbors for each vertex
for vertex in range(num_vertices):
    graph.find_neighbors(vertex)

```

```
Before adding connections:
Adjacency List:
0:
1:
2:
3:
4:

After adding connections:
Adjacency List:
0: 3 1
1: 4 2
2:
3: 4 1
4: 2

Neighbors of vertex 0: 3,1

Neighbors of vertex 1: 4,2

Neighbors of vertex 2:
```

```
Neighbors of vertex 3: 4,1

Neighbors of vertex 4: 2
```

Adjacency list for Directed weighted graph

```
class WEdge {
    int destination;
    int weight;
    WEdge next;

    public WEdge( int destination, int weight) {
        this.destination = destination;
        this.weight = weight;
        this.next = null;
    }
}
```

```
public class GraphDirectedWeightedAdjacencyList {

    private int numVertices;     // Total number of vertices in the graph
    private WEdge[] adjList;     // Array of linked lists (Adjacency List)

    // Constructor: initialize graph with given number of vertices
    public GraphDirectedWeightedAdjacencyList(int numVertices) {
        this.numVertices = numVertices;
        adjList = new WEdge[numVertices]; // Initially all entries are null
    }

    // Add multiple edges from a 2D array
    public void addEdges(int[][] edges) {
        for (int i = 0; i < edges.length; i++) {
            int source = edges[i][0];       // Starting vertex
            int destination = edges[i][1];  // Ending vertex
            int weight = edges[i][2];       // Weight of the edge

            // For DIRECTED graph:
            // Add edge in only 1 direction
            addEdge(source, destination, weight); // Add edge to graph
        }
    }

    // Add a single directed weighted edge
    public void addEdge(int source, int destination, int weight) {

        // Step 1: Create a new edge node
        WEdge newEdge = new WEdge(destination, weight);

        // Step 2: Link new edge to the beginning of the list
        // (prepend for O(1) insertion)
        newEdge.next = adjList[source];

        // Step 3: Update head of the list for this source vertex
        adjList[source] = newEdge;
    }
```

```java
// Print the full adjacency list
public void printList() {
    System.out.println("Adjacency List with Weights:");

    // Traverse each vertex
    for (int i = 0; i < numVertices; i++) {
        System.out.print(i + ": ");

        // Traverse linked list of edges for vertex i
        WEdge current = adjList[i];
        while (current != null) {

            // Print (destination, weight)
            System.out.print("(" + current.destination + ", " + current.weight + ") ");

            current = current.next; // Move to next edge
        }
        System.out.println();
    }
}

// Print neighbors of a specific vertex
public void findNeighbors(int vertex) {

    System.out.print("\nNeighbors of vertex " + vertex + ": ");

    WEdge current = adjList[vertex];

    // Traverse all edges from this vertex
    while (current != null) {

        // Print neighbor and weight
        System.out.print("(" + current.destination + ", " + current.weight + ")");

        // Add comma if not the last element
        if (current.next != null) {
            System.out.print(",");
        }

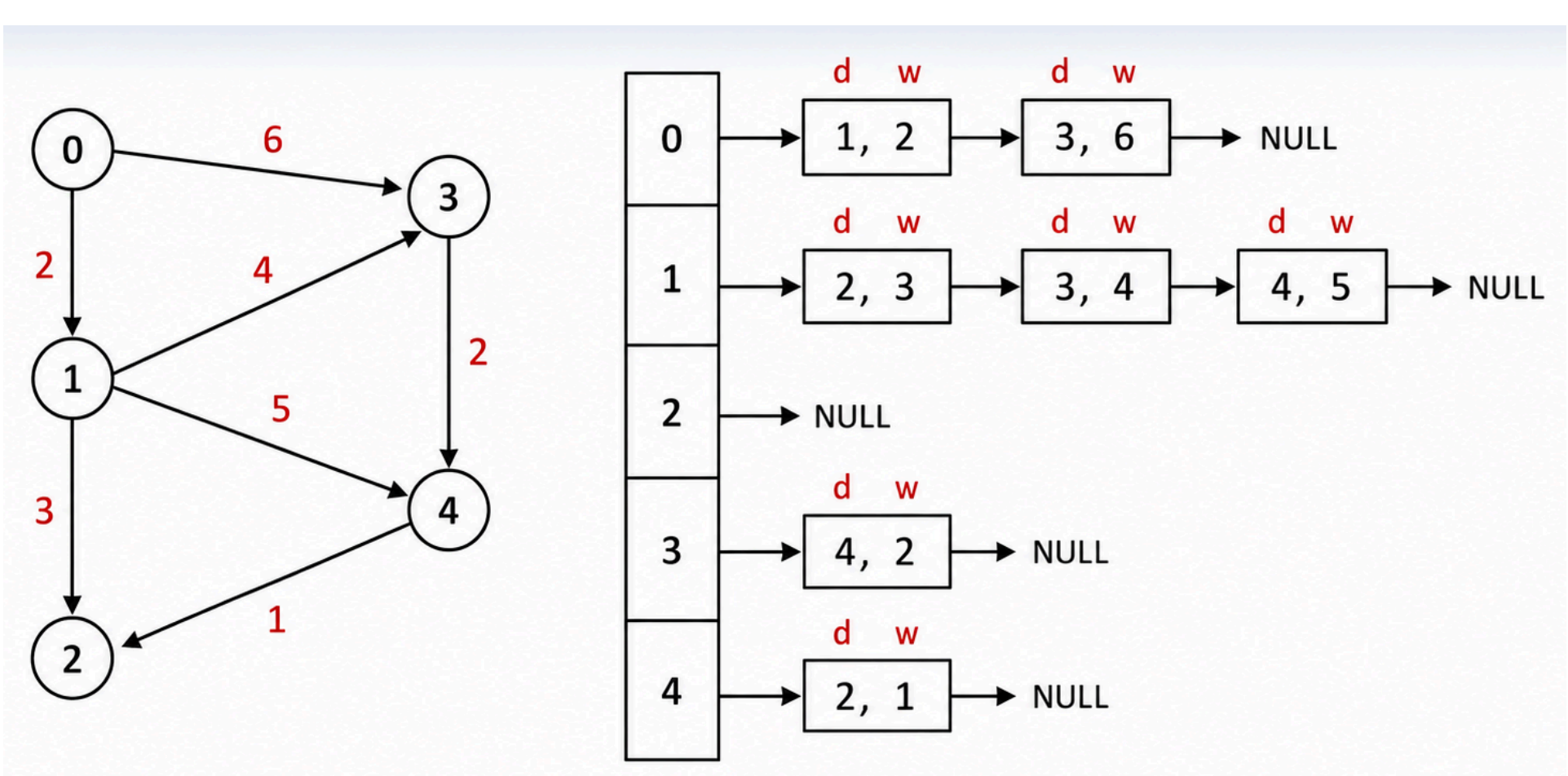
        current = current.next; // Move to next edge
    }

    System.out.println();
}
```

Python

↳ 1 cell hidden

Driver



```java
// Driver code
public static void main(String[] args) {
    int numVertices = 5;
    int[][] edges = {
        {0, 1, 2}, {0, 3, 6},
        {1, 2, 3}, {1, 4, 5},
        {3, 1, 4}, {3, 4, 2},
        {4, 2, 1}
    };

    GraphWeightedAdjacencyList graph = new GraphWeightedAdjacencyList(numVertices);

    System.out.println("Before adding connections:");
    graph.printList();

    graph.addEdges(edges);

    System.out.println("\nAfter adding connections:");
    graph.printList();

    for (int vertex = 0; vertex < numVertices; vertex++) {
        graph.findNeighbors(vertex);
```

```python
# Driver code
num_vertices = 5
edges = [
    (0, 1, 2), (0, 3, 6),   # (u, v, weight)
    (1, 2, 3), (1, 4, 5),
    (3, 1, 4), (3, 4, 2),
    (4, 2, 1)
]

# Create an instance of the AdjacencyListGraph
graph = AdjacencyListGraph(num_vertices)

print("Before adding connections:")
graph.print_list()

# Add weighted directed edges
graph.add_edges(edges)

print("\nAfter adding connections:")
graph.print_list()

# Print neighbors for each vertex
for vertex in range(num_vertices):
    graph.find_neighbors(vertex)

```

```
Before adding connections:
Adjacency List with Weights:
0: s= 0
1: s= 1
2: s= 2
3: s= 3
```